End-to-End MLOps Pipeline for Heart Disease Risk Prediction

Machine Learning Operations (MLOps) — AIMLCZG523 — Assignment 01

Abhishek Singh

October 2026

Introduction and Problem Statement

This report documents an end-to-end Machine Learning Operations (MLOps) solution for predicting the presence of heart disease from clinical attributes in the UCI Heart Disease dataset. The work follows a production-style workflow: reproducible data acquisition, exploratory analysis, model development with cross-validation, experiment tracking, automated testing, containerized serving, Kubernetes deployment, and API monitoring.

The modelling problem is binary classification. The raw UCI target uses values 0 through 4, where 0 denotes absence of disease and 1–4 denote presence at increasing angiographic severity. For this assignment the target is binarized to **0 = absent** and **1 = present**. The served model is an educational demonstration only and is **not intended for medical diagnosis or treatment**.

Code repository: https://github.com/Maverick2509/AI-Projects

Local API (Docker Compose): `http://127.0.0.1:8000`
Kubernetes (Minikube NodePort tunnel): `http://127.0.0.1:51430`
MLflow UI: `http://127.0.0.1:5001`
Prometheus: `http://127.0.0.1:9090`
Grafana: `http://127.0.0.1:3000` (default login `admin` / `admin`)

Architecture Overview

The system is organized as a linear MLOps pipeline. Source code lives in GitHub. Every push runs GitHub Actions, which lints the code, executes unit tests, downloads and prepares data, trains models, and uploads training artifacts. Training writes parameters, metrics, and sklearn pipelines to MLflow. The selected preprocessing-plus-estimator pipeline is packaged as a Joblib artifact and loaded by a FastAPI service. That service is containerized with Docker, deployed to Minikube, and scraped by Prometheus; Grafana is provisioned with a dashboard for request rate, server errors, and latency.

Technology stack

Layer	Choice	Role in this project
Language	Python 3.13	Training, tests, and API
Data	pandas, ucimlrepo	Download and cleaning
Models	scikit-learn	Logistic Regression and Random Forest

Layer	Choice	Role in this project
Tracking	MLflow + SQLite	Parameters, metrics, model artifacts
API	FastAPI + uvicorn	`/health`, `/predict`, `/metrics`
Quality	Pytest, Ruff	Unit tests and lint in CI
CI/CD	GitHub Actions	Lint, test, train, upload artifacts
Containers	Docker Compose	API + Prometheus + Grafana
Orchestration	Kubernetes / Minikube	Deployment, Service, probes
Monitoring	Prometheus, Grafana	Request rate, errors, p95 latency

Setup and Reproducible Execution

The project is designed to run from a clean checkout using `requirements.txt`.

```
git clone https://github.com/Maverick2509/AI-Projects.git && cd AI-Projects
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m src.data.download && python -m src.data.prepare && python -m src.eda && python -m src.
python -m pytest -q
# API locally:  uvicorn api.main:app --host 127.0.0.1 --port 8000
# Docker:       docker compose -f deployment/k8s/docker-compose.yml up -d --build
# Kubernetes:   kubectl apply -f deployment/k8s/ && minikube service heart-disease-api
```

A sample prediction payload accepted by `POST /predict` is:

```
{
  "features": {
    "age": 62, "sex": 0, "cp": 4, "trestbps": 160, "chol": 164,
    "fbs": 0, "restecg": 2, "thalach": 145, "exang": 0,
    "oldpeak": 6.2, "slope": 3, "ca": 3, "thal": 7
  }
}
```

Data Acquisition and Exploratory Data Analysis

Dataset and download

The dataset is obtained programmatically from the UCI Machine Learning Repository (id 45) in `src/data/download.py` using `ucimlrepo.fetch_ucirepo`. Features and the target are concatenated into `data/raw/heart_disease.csv`. This avoids hand-copied CSVs and keeps the acquisition step reproducible in CI.

The downloaded table contains **303 records and 14 columns**: age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, and target. The raw target distribution is 164 / 55 / 36 / 35 / 13 for values 0, 1, 2, 3, and 4 respectively. Missingness is limited to `ca` (4 values) and `thal` (2 values). There are no duplicate rows in the downloaded file.

Cleaning and preprocessing

`src/data/prepare.py` performs the following steps:

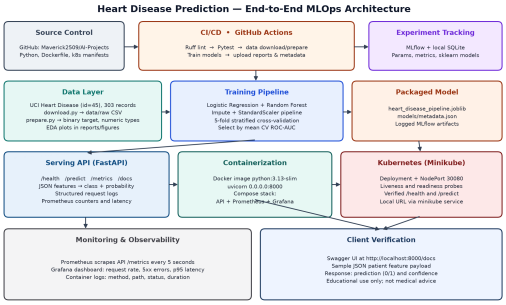


Figure 1: Figure 1. Architecture spanning source control, CI/CD, data and training, packaging, serving, Kubernetes, and monitoring.

1. Replace common missing-value markers (`?`, `NA`, `N/A`, empty string) with NaN.
2. Drop duplicate rows.
3. Coerce the target to numeric, drop rows without a target, and binarize (`target > 0`).
4. Coerce remaining feature columns to numeric, leaving true missing values in place so that the training pipeline can impute them.

The cleaned file is written to `data/processed/heart_disease_clean.csv`. After binarization the class counts are **164 absent (0)** and **139 present (1)** — moderately balanced, so accuracy remains a useful metric, but class-weighted estimators were still used.

Median imputation and standard scaling are **not** applied in the CSV cleaning step. They live inside a scikit-learn `Pipeline` so that the same transformations are fitted on training folds and applied at inference time.

Exploratory visualizations

EDA is implemented in `src/eda.py` and writes figures to `reports/figures/`.

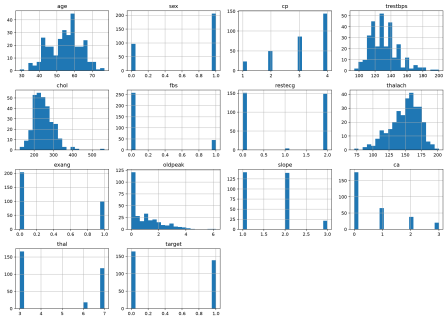


Figure 2: Figure 2. Feature histograms. Age is concentrated between 40 and 70. Resting blood pressure and cholesterol are right-skewed. Several clinical attributes (`sex`, `fbs`, `exang`, `cp`, `slope`, `thal`) are discrete.

The class-balance plot (164 absent vs 139 present) is stored in `reports/figures/class_balance.png`. These plots informed two modelling choices. First, mixed continuous and coded-categorical features are all treated as numeric inputs and standardized; one-hot encoding was not required because UCI already encodes the categoricals as numbers. Second, `class_weight="balanced"` was enabled because the positive class is slightly smaller and missed disease cases are more costly in this domain.

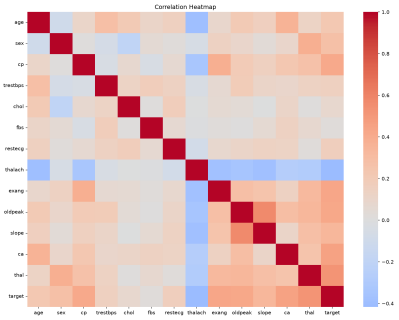


Figure 3: Figure 3. Correlation heatmap. The target is positively associated with chest-pain type (`cp`), exercise-induced angina (`exang`), ST depression (`oldpeak`), number of vessels (`ca`), and thalassemia (`thal`). Maximum heart rate (`thalach`) is negatively associated with the target.

Feature Engineering and Model Development

Feature pipeline

All 13 clinical attributes are retained. The training script builds a `ColumnTransformer` that applies median imputation and `StandardScaler` to every feature, then attaches a classifier. The full object is a scikit-learn `Pipeline`, so serialization stores preprocessing and the estimator together.

A stratified 80/20 train-test split (`random_state=42`) holds out a test set. Model selection uses **5-fold stratified cross-validation on the training split only**. The selection metric is mean CV ROC-AUC, which is threshold-independent and appropriate for a slightly imbalanced medical classification task. Held-out test metrics are reported for honesty but are not used to pick the winner.

Models and hyperparameters

Two classifiers were trained:

- **Logistic Regression** — linear, interpretable baseline; `max_iter=2000`, `class_weight="balanced"`, default `C=1.0`.
- **Random Forest** — non-linear ensemble; `n_estimators=300`, `min_samples_leaf=2`, `class_weight="balanced"`.

These are complementary: logistic regression tests whether a linear decision surface is sufficient after scaling; random forest tests whether interactions and non-linear splits improve discrimination. Limited tree depth growth was controlled via `min_samples_leaf=2` rather than an aggressive grid search, which is more appropriate for a 242-row training split.

Evaluation results

Model	CV Acc.	CV Prec.	CV Rec.	CV ROC-AUC	Test Acc.	Test Prec.	Test Rec.	Test ROC-AUC
Logistic Regression	0.814	0.815	0.774	**0.896**	0.869	0.813	0.929	0.950
Random Forest	0.802	0.800	0.765	0.887	0.885	0.839	0.929	0.962

Logistic Regression is selected because of the higher mean CV ROC-AUC (0.896 vs 0.887), even though Random Forest is slightly stronger on the 61-row test set. Recall on the test set is 0.929 for both models, which is desirable: false negatives (missed disease) are more serious than false positives in a screening-style API. Test ROC-AUC values above 0.95 should be read cautiously because the test split is small; the CV estimates around 0.89 are the more trustworthy generalization numbers.

The selected pipeline is saved to `models/heart_disease_pipeline.joblib`, with metadata in `models/metadata.json` recording the selected model name, feature column order, and selection score.

Experiment Tracking

MLflow is configured against a local SQLite store (`mlflow.db`) under the experiment name `heart-disease-classification`. Each estimator is a named run. The training script logs:

- parameters (`model_name`, `random_state`, `cv_folds`, and the estimator's `get_params()`);
- metrics (CV and test accuracy, precision, recall, ROC-AUC);
- a scikit-learn model artifact for each run.

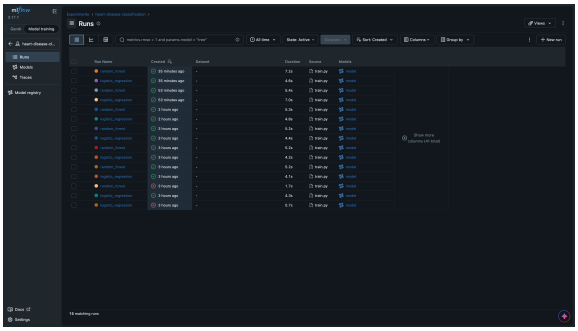


Figure 4: Figure 4. Repeated training runs for `logistic_regression` and `random_forest`, each logging a model artifact from `train.py`.

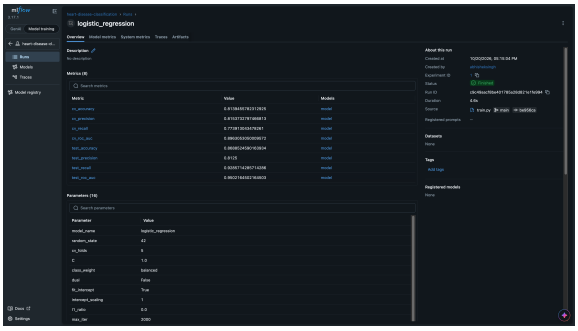


Figure 5: Figure 5. Detail view of the selected Logistic Regression run: CV ROC-AUC 0.896, test recall 0.929, `class_weight=balanced`, `max_iter=2000`, 5 CV folds.

The UI is started with:

```
mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5001
```

Port 5001 is used because macOS AirPlay occupies port 5000.

Model Packaging and Reproducibility

Reproducibility is enforced at three layers:

1. **Pinned dependencies.** `requirements.txt` records the exact environment used for training and serving (notably `scikit-learn==1.9.1`, `mlflow==3.17.1`, `fastapi==0.151.0`).
2. **Single inference artifact.** `joblib.dump` stores the entire `Pipeline` (imputer, scaler, classifier). The API never reimplements preprocessing.
3. **Feature contract.** `models/metadata.json` lists the 13 feature names in training order. The API rejects requests with missing or unexpected keys and returns HTTP 422, which prevents silent column misalignment.

MLflow additionally stores sklearn model artifacts per run, providing a second packaging path if a run needs to be reloaded later.

CI/CD Pipeline and Automated Testing

Unit tests

`tests/test_data.py` covers the cleaning contract:

- multi-valued UCI targets become binary `{0, 1}`;
- duplicate rows are removed;
- `"?"` in a feature becomes NaN so that the imputer can handle it;
- a missing `target` column raises `ValueError`.

Pytest runs locally and in CI (`python -m pytest -q`). Ruff is the linter (`ruff check .`). The GitHub Actions job is configured to fail the workflow when lint or tests fail, which was observed on early commits before the pipeline turned green.

GitHub Actions

Workflow file: `.github/workflows/ci.yml`. Trigger: push, pull request, and `workflow_dispatch`. Job `test-and-train` on `ubuntu-latest`:

1. Checkout and set up Python 3.13 with pip caching.
2. Install `requirements.txt`.
3. Lint with Ruff.
4. Run Pytest.
5. Download and prepare the dataset.
6. Train models (`python -m src.train`).
7. Upload `reports/`, `models/metadata.json`, and `mlruns/` as the `training-results` artifact (retained 14 days), including on failure (`if: always()`).

Successful run URL: https://github.com/Maverick2509/AI-Projects/actions/runs/39419677107

The red-then-green history is evidence that the pipeline does not report success when code or tests break.

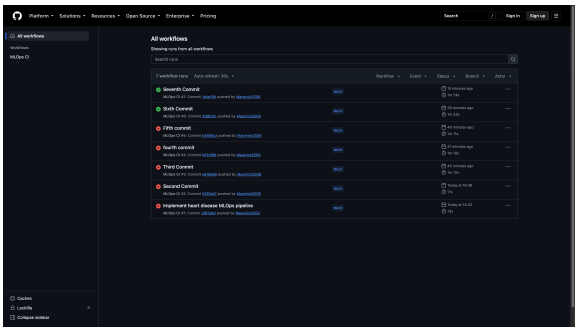


Figure 6: Figure 6. Workflow history. Early commits failed (the pipeline correctly stopped on errors). The latest two runs, including **Seventh Commit**, succeeded.

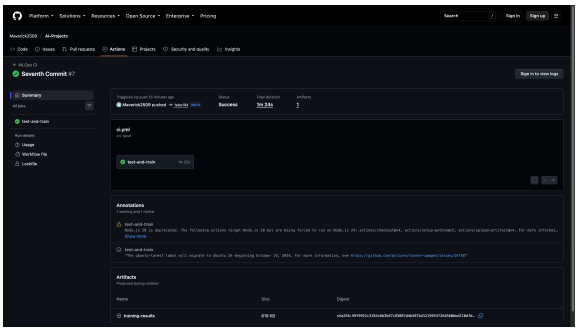


Figure 7: Figure 7. Seventh Commit (#7) completed successfully in 1 m 24 s. The test-and-train job ran 1 m 22 s and published the training-results artifact (619 KB).

Model Containerization

The `Dockerfile` uses `python:3.13-slim`, installs pinned requirements, copies `src/`, `api/`, and `models/`, exposes port 8000, and starts `uvicorn api.main:app`. Docker Compose (`deployment/k8s/docker-compose.yml`) builds this image and runs it beside Prometheus and Grafana; `docker ps` showed `heart-disease-api` on host port 8000, Prometheus on 9090, and Grafana on 3000.

The API exposes:

- `GET /health` — liveness/readiness style status and `model_loaded`;
- `POST /predict` — JSON feature map, returning `prediction`, `probability` (positive-class confidence), `model_name`, and a medical-use disclaimer;
- `GET /metrics` — Prometheus metrics;
- `GET /docs` — Swagger UI.

Structured request logs are emitted for every call (`method`, `path`, `status`, `duration_seconds`) and are visible in `docker logs heart-disease-api`.

Production Deployment on Kubernetes

The API is deployed to a local Kubernetes cluster using Minikube. Manifests:

- `deployment/k8s/deployment.yml` — one replica, image `heart-disease-api:latest`, `imagePullPolicy: IfNotPresent`, container port 8000, HTTP liveness and readiness probes on `/health`, CPU/memory requests and limits.

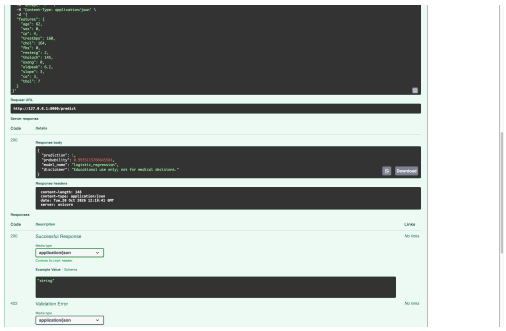


Figure 8: Figure 8. Sample inference through Swagger. Request URL `http://127.0.0.1:8000/predict` returned HTTP 200 with `prediction: 1`, `probability: 0.9935`, and `model_name: logistic_regression`.

- `deployment/k8s/service.yml` — NodePort service mapping port 8000 to node port **30080**.



Figure 9: Figure 9. `kubectl get` output. Pod `heart-disease-api-...` is Ready 1/1. Service type is NodePort `8000:30080`.

After `minikube service heart-disease-api`, the API was verified at `http://127.0.0.1:51430`. `GET /health` returned `{"status":"ok","model_loaded":true}` and `POST /predict` returned the same class and probability as the Docker service. Swagger for the Kubernetes endpoint is available at `http://127.0.0.1:51430/docs`.

Monitoring and Logging

Three observability mechanisms are in place.

Application logs. FastAPI middleware records method, path, status, and duration. Prediction handlers log the predicted class. Unhandled exceptions are logged with stack traces.

Prometheus metrics. Counters `api_requests_total` and `api_server_errors_total`, plus histogram `api_request_duration_seconds`, are exposed on `/metrics`. Prometheus scrapes `api:8000` every 5 seconds (`deployment/k8s/prometheus.yml`).

Grafana dashboard. Provisioned datasource points at Prometheus. Dashboard *Heart Disease API Monitoring* (uid `heart-disease-api`) plots request rate by status, 5xx error rate, and p95 latency, with a 5-second refresh.

Grafana is available at `http://127.0.0.1:3000` (anonymous first login `admin`/`admin`; Grafana will prompt to change the password). The provisioned dashboard lives under folder **MLOps Assignment**.

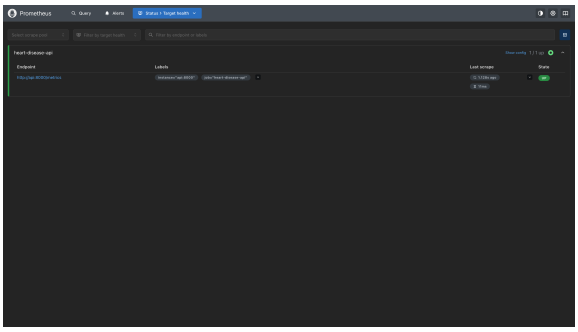


Figure 10: Figure 10. Prometheus target health: job `heart-disease-api`, endpoint `http://api:8000/metrics`, state **UP**.

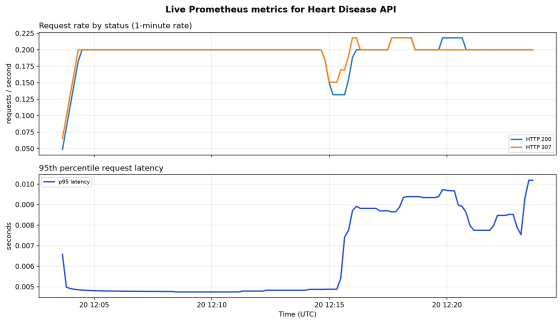


Figure 11: Figure 11. Live Prometheus queries over a 20-minute window after exercising `/health`, `/predict`, and `/docs`. Request traffic is visible; p95 latency remains well below 50 ms. No 5xx series was recorded in this window.

Production-Readiness Checklist

Requirement	Evidence
Scripts run from a clean `requirements.txt` setup	Documented in Section 3; CI installs the same file
Model serves in an isolated Docker environment	Figure 8; `/predict` HTTP 200 from the container
Pipeline fails on code or test errors	Figures 6–7: failed early commits, green latest run
`/predict` returns class and confidence	Figure 8
Deployment manifest and live endpoint	Figure 9; Minikube Service `8000:30080`
Request logging and metrics	Figures 10–11

Conclusions

This assignment implemented a complete MLOps path from UCI data download to a monitored, containerized API. Logistic Regression was selected on cross-validated ROC-AUC (0.896) and packaged with its preprocessing pipeline so that training and serving cannot drift apart. GitHub Actions enforces linting, tests, and retraining on every push. The same image runs under Docker Compose and Minikube, with Prometheus confirming a healthy scrape target and sub-50 ms p95 latency under sample traffic.

Remaining operational notes: the current Kubernetes Service is NodePort rather than a cloud LoadBalancer, which is appropriate for Minikube but would be replaced by an Ingress or cloud LB in a public deployment. The held-out test set is small (61 rows), so CV metrics should remain the promotion criterion. The API is an academic prototype and must not be used for clinical decisions.

Repository: https://github.com/Maverick2509/AI-Projects

Local testing: `docker compose -f deployment/k8s/docker-compose.yml up -d --build`, then `curl http://127.0.0.1:8000/health` and Swagger at `/docs`. Prometheus: `http://127.0.0.1:9090/targets`. Grafana: `http://127.0.0.1:3000` (`admin`/`admin`). Kubernetes: `kubectl apply -f deployment/k8s/` and `minikube service heart-disease-api`. Attach a short screen recording of this walkthrough as required by the assignment deliverables.